Models Take Notes at Prefill: KV Cache Can Be Editable and Composable

Anonymous Author(s)

Preprint. Under review.

Abstract

Prefix caching reuses a transformer's prefill only across an exact shared prefix, so a single changed field—a clock tick, a session id, an account-status flip—invalidates the entire downstream cache. We begin from a puzzle: the cache region *before* a field is provably reusable (key/value deviation 0.0), yet surgically overwriting only the field's own key/value vectors and reusing the rest leaves the model acting on the *old* value. We explain why, and turn the explanation into two capabilities. Through four independent causal probes (replicated across five model families) we show that at prefill the model computes the *field-conditioned conclusion* and stores it on downstream aggregator/delimiter tokens: the field's own key/value drives less than 1% of the decision, while the downstream "notes" drive essentially all of it. The KV cache is thus a notebook of memoized conclusions, and two consequences follow. **(i) It is editable:** because the conclusion is already written downstream, the right fix is not recomputation but a salient *erratum* that amends the notes; `field+erratum` matches the strong hoist-to-end baseline without rewriting the prompt, and under chain-of-thought the cheap in-place edit alone recovers the oracle decision (0.94 at 8B) at $\sim$1% of the compute. **(ii) It is composable:** the notes are localized, position-portable, and context-robust, so a precompiled skill can be RoPE-repositioned and spliced into any context—behaviorally indistinguishable from full recompute (logit cosine 0.90–0.999 across twelve models) at $O(L)$ rather than $O(L^2)$ time-to-first-token (13.9$\times$ at 32k tokens). A keystone experiment—editing a field *inside* a transplanted skill—reproduces the editing mechanism verbatim, showing edit and compose act on one substrate; a unified agent over 10 domains $\times$ 10 instances (300 decisions) is decision-identical to full recompute (agreement 0.81–1.0) at up to 14.9$\times$ lower latency. The substrate is any per-token attention KV cache: we validate it across scale, quantization, Mixture-of-Experts, multimodal image caches, and—via small adapters—Multi-head Latent Attention and sliding-window and interleaved-MRoPE models, and we map exactly where it holds up to the 2026 sparse-attention frontier. The caching machinery itself is prior art; our contributions are the mechanism, the unification, a decision-governance evaluation, and the attention-variant adapters.

1 Introduction

Modern LLM agents re-read long, mostly-static instructions on every turn: a system policy, a tool specification, retrieved documents. Key/value (KV) caching makes this affordable by reusing the prefill computation across turns—but only across an *exact* shared prefix. The moment a single token changes inside the reused region—a timestamp, a user id, an order's status—the keys and values of *every* subsequent token are invalidated, because each attended to the token that changed. The de-facto workaround is to *hoist* all mutable content to the end of the prompt so the static prefix

stays cache-aligned. This pushes an inference-layer constraint into the application layer: fields referenced in several places, nested sub-agent prompts, and dynamically assembled contexts cannot all be cleanly hoisted, and the application must enumerate every mutable field in advance.

This paper starts from a concrete puzzle. The region of the cache *before* a field is, by construction, independent of the field's value—we measure a key/value deviation of exactly 0.0 when the field changes. One might therefore hope to *surgically* refresh only the field's own keys and values, leave the rest of the cache stale, and pay almost nothing. We find this fails completely: the model's decision reverts to the *old* field value, as if the edit never happened (Figure 1b).

The discovery. The reason, which we establish causally, is that transformers do not defer their reasoning to decode time. At *prefill* the model already computes the *field-conditioned conclusion* and writes it onto downstream tokens—disproportionately onto aggregator/delimiter tokens that later positions attend through. The decision then reads these *notes*, not the field itself: across models the field's own KV causally drives less than 1% of the decision, while the downstream notes drive essentially all of it. The KV cache is best understood not as a frozen byproduct of prefill but as a *notebook of memoized conclusions* (Figure 1a).

Two capabilities from one mechanism. This reframing is generative. (1) If the conclusion is already written downstream, then *editing* a field means amending the notes, not recomputing them: a one-line salient *erratum* suffices, and we map exactly when the cheap in-place edit alone is enough (under reasoning) versus when it is not. (2) If the notes are localized and position-portable, then a reusable skill can be *composed* into a new context by repositioning and splicing its cached notes—no recompute. A *keystone* experiment, editing a field *inside* a transplanted skill, shows the two operations act on a single substrate (Figure 2).

Contributions.

- **A mechanism** (Section 3): *attention-mediated memoized inference*, established by four independent causal probes (locality patching, suffix concentration, linear probing, circuit knockout), plus content/conclusion dissociation, layer-timing, specificity, and note-injection controls (Appendix E)—replicated across *five model families* (Qwen3, Llama-3.1, Gemma-2, Mistral)—connecting to delimiter-token aggregation observed in interpretability.
- **An editing capability** (Section 4): naive KV editing fails; the erratum / `field+erratum` fix matches the hoist-to-end oracle without prompt surgery; a reasoning/scale analysis of when the $\sim$1%-compute in-place edit suffices; and a head-to-head with *weight* editing (ROME, LoRA) showing it is the wrong tool for mutable per-request state (global contamination, collateral, 30–50$\times$ slower).
- **A composing capability** (Section 5): position-portable transplant of precompiled skills, $O(L)$ vs. $O(L^2)$ TTFT (13.9$\times$ at 32k), with a seam-repair knob—honestly credited to prior caching work, with our addition being the mechanism that explains it and a correctness lens.
- **The unification** (Section 6): the keystone and a unified edit+compose agent over twelve models.
- **Reach and scope** (Section 7): validation across scale, MoE, quantization, and multimodal image caches; small adapters for MLA and interleaved M-RoPE; a fix for sliding-window; and an analysis of where the substrate holds up to the 2026 sparse/compressed-attention frontier.
- **Systems payoff** (Section 8): a real agentic environment and a comprehensive online vLLM serving benchmark (V1 engine, continuous batching, Poisson load)—98.5% vs. 1% prefix-cache hit-rate, 53–398$\times$ lower $p90$ TTFT, and throughput gains that grow with load to 14.5$\times$.

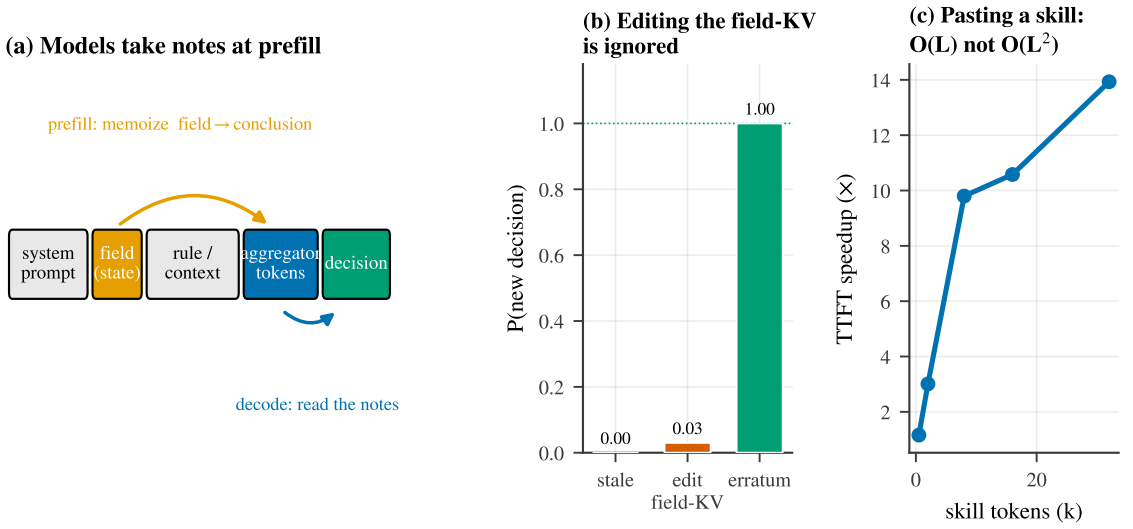

Figure 1: **Models take notes at prefill.** (a) At prefill the model memoizes the field-conditioned conclusion onto downstream aggregator tokens (orange); at decode the decision reads those notes (blue). (b) Consequently, surgically editing the field's own KV is ignored—the fix is an *erratum* that amends the notes. (c) Because the notes are position-portable, a precompiled skill can be pasted into a new context in $O(L)$ rather than $O(L^2)$ time.

- **An application: user memory** (Section 9): the large, mutable user-memory document is both composed (precompiled, repositioned, spliced) and edited (in-place / erratum) as one substrate—decision-faithful to full recompute at 2.3–4.3× lower time-to-first-token, validated to 70B and on real long-conversation memory (LoCoMo, transplant ≡ full recompute in QA accuracy)—with a pre-registered, statistically-controlled evaluation.

2 Related work

Where computation is stored, and how to edit it. A line of interpretability work localizes stored *knowledge* in transformer weights and edits it there: ROME and MEMIT [15, 16] locate and rewrite factual associations in MLP weights, while circuit analyses such as the indirect-object-identification study [23] trace how specific computations are carried by attention heads. Weight editing targets *durable, global* facts; we compare against a faithful ROME and a LoRA fine-tune empirically (Table 1) and find them ill-suited to *mutable per-request* state—a global edit contaminates concurrent requests and damages unrelated decisions—which is exactly the niche the editable KV cache fills. Closest in spirit, Lindsey et al. [10] find models commit *plans* to specific tokens (e.g. a planned rhyme stored on a line-break token) during the forward pass. We study a complementary object: not weights and not decode-time computation, but the *KV cache*—the activations an inference system already stores and an editor can directly manipulate—and show it holds *memoized conclusions* concentrated on aggregator/delimiter tokens. To our knowledge this is the first causal account of why in-place KV field editing fails and what to do instead.

KV reuse and composable caching. Reusing precomputed KV beyond an exact prefix is an active systems topic, and our *composing* capability builds directly on it; we claim none of the caching machinery as novel. Prompt Cache [4] precomputes reusable prompt modules with position placeholders and splices them. CacheBlend [26] reuses non-prefix chunk KV and selectively recom-

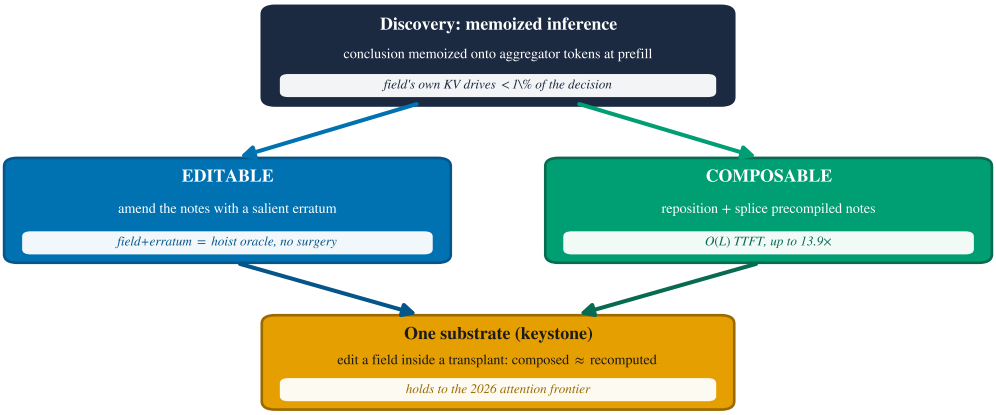

Figure 2: **One mechanism, two capabilities, one substrate.** The discovery that models memoize field-conditioned conclusions at prefill makes the KV cache both *editable* (amend the notes with a salient erratum) and *composable* (reposition and splice precompiled notes); a keystone experiment shows they act on a single substrate, which holds up to the 2026 attention frontier.

putes ∼15% of tokens to restore cross-attention. EPIC [5] introduces position-independent caching with ATTNLINK, recomputing only a few chunk-boundary tokens (exploiting attention sinks) for near-linear recompute. CacheSlide [11] reuses KV in a position-*aware* way via relative-position-dependent caching, and MPIC [30] extends position-independent caching to the multimodal setting by recomputing image-boundary tokens; KVLink [25] is a further reuse system. Mapped onto our terms, our RoPE-repositioning is CacheSlide's relative-position reuse, our seam-repair is the boundary recompute of CacheBlend/EPIC/MPIC, and our image-KV transplant is MPIC's idea (we *re-rotate* M-RoPE rather than recompute). Our contributions over this line are orthogonal: (i) the *mechanism* that explains *why* boundary recompute is what is needed; (ii) a *decision-governance* evaluation—does a transplanted skill still *govern the tool decision*, rather than only preserve perplexity or throughput; (iii) the *editing* axis and the keystone unification; and (iv) adapters that extend the operations to new attention representations (MLA, interleaved M-RoPE, sliding-window).

Prefix caching, KV compression, and reuse systems. Production prefix caching (vLLM Automatic Prefix Caching, SGLang RadixAttention) reuses exact prefixes. A large literature instead *compresses* or *evicts* the cache: StreamingLLM keeps attention sinks [24]; H2O [29], Scissorhands [13], and SnapKV [9] evict low-importance tokens; Quest [20] keeps all tokens but attends sparsely. Serving systems stream or share cached KV across requests—CacheGen [12] for fast loading and RAGCache [7] for retrieval reuse. These change *which* tokens are present, and compose with our edit/transplant operations only over retained tokens; we treat them, the latent/decoupled-RoPE representation of Multi-head Latent Attention [2, 3], and the 2026 sparse/compressed-attention designs as scope boundaries in Section 7. Our repositioning relies on rotary position embeddings [19] and their extensions [17].

Activation-level interventions. Beyond weight editing, a body of work intervenes on *activations*: steering and inference-time interventions [8], task and function vectors [6, 21], and the causal-mediation/activation-patching methodology we adapt [22]. These edit residual-stream directions to change behavior; we instead read and write the *KV cache* itself—the per-token activations a serving system already persists—which is what makes the intervention both interpretable and deployable.

Agents and tool use. Our evaluation targets tool-using agents in the style of ReAct [27] and Toolformer [18], and we measure end-to-end task success on the τ-bench tool-agent-user benchmark [28], where state changes mid-trajectory make cache staleness a first-class correctness problem.

3 The discovery: memoized inference in the KV cache

Setup. We study a minimal but agent-realistic decision: a context contains a policy rule and a *mutable field* whose value determines the correct action (e.g. "cancel an order only if its status is `pending`"; with `status=shipped` the correct action flips from *cancel* to *deny*). After prefilling the full context we compare four cache states at the decision token: *stale* (old field, old downstream), *field-only* (refresh the field's KV, leave the downstream stale), *full-downstream* (recompute everything after the field), and *oracle* (a clean prefill of the new value). We report *decision recovery*: the fraction of the oracle's flip that a cache state reproduces (0 = behaves like stale, 1 = behaves like oracle). Four independent probes converge on one account (Figure 3).

(1) Locality: the field's own KV barely matters. Refreshing only the field's KV recovers essentially none of the decision—field-only recovery is -0.028 on Llama-3.1-8B and near zero across models—whereas recomputing the downstream recovers it fully (1.0) (Figure 3a). The field is read *indirectly*: its causal contribution to the decision is under 1%.

(2) Suffix concentration: the effect lives downstream and late. Sweeping how many downstream tokens we patch (ranked by causal effect) shows recovery accrues slowly and saturates only as we include many tokens after the field, with the causal mass concentrated in mid/late layers (Figure 3b). The information the decision needs is not in the field but distributed over the tokens that followed it at prefill (Figure 3c).

(3) Linear decodability. The field-conditioned conclusion is linearly decodable from those downstream tokens' residual stream at prefill time—i.e. the model has already *computed and written down* the answer, not merely copied the field.

(4) Knockout and dose-response. Ablating the high-effect downstream tokens flips the decision back to stale, and the effect grows with the number/position of memoizing tokens, ruling out a diffuse explanation: specific aggregator/delimiter positions carry the conclusion. This mirrors the delimiter-token aggregation reported by Lindsey et al. [10] for forward planning; here the stored quantity is a backward-looking, field-conditioned *conclusion*.

What the note contains. If the note were a verbatim copy of the field, restating the value would suffice and emphatic wording would be inert. Instead, a wording ablation (Figure 3d) shows that once the corrected value is present, the override phrasing is *redundant* (bare value, tagged update, and explicit override all reach ≈ 1.0 P(safe)), while aggressive "disregard your earlier conclusion and re-evaluate" phrasing actively *hurts* (0.81). The note behaves like a committed conclusion that a late, salient correction can overwrite—but that confrontational instructions can destabilize. We name the phenomenon **attention-mediated memoized inference**, and the rest of the paper shows it makes the cache both editable (Section 4) and composable (Section 5).

Separating conclusion from content—causal, temporal, and writable. Four further controls (three models each: Qwen3-8B/4B, Llama-3.1-8B; Appendix E) close the gap between "the conclusion is *decodable* downstream" and "the decision *uses* a memoized conclusion." **(i) Dissociation.** We hold the field value byte-identical and flip a single rule token (a polarity *trigger*) so the *conclusion* inverts while the *content* does not; transplanting the downstream notes alone then carries the whole flipped conclusion (recovery 0.998–1.009) while patching the differing rule token carries none (-0.007 to $+0.007$)—the decision cannot be re-encoding field content, since the con-

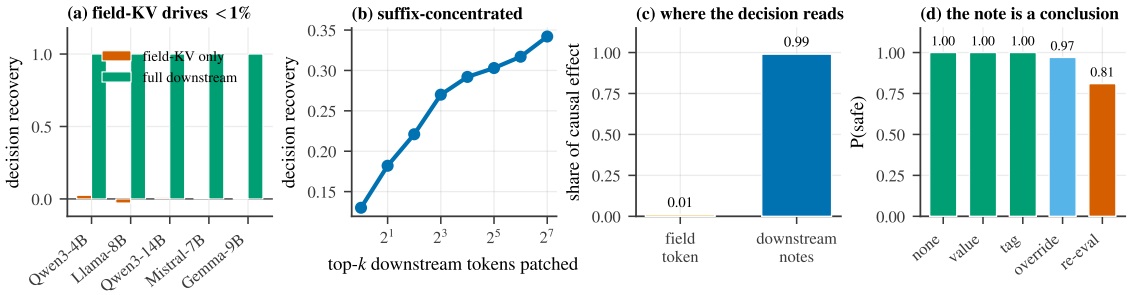

Figure 3: **Four causal probes for memoized inference.** (a) Refreshing the field's own KV recovers ≈ 0 of the decision; recomputing the downstream recovers it fully. (b) Recovery is suffix-concentrated, accruing only as many post-field tokens are patched. (c) The decision reads almost entirely from downstream notes, not the field token. (d) Once the value is present, override wording is redundant and "re-evaluate" phrasing hurts—the note is a committed conclusion.

tent is constant. (A linear probe finds *both* conclusion and field identity decodable downstream, so decodability alone cannot separate them: the causal patch is the necessary instrument.) **(ii) Timing.** Within the single prefill pass, the conclusion becomes linearly decodable on the downstream aggregator at depth 0.31–0.39, roughly twelve layers *before* the decision token's logit-lens commit (depth 0.73–0.77): the note is written at prefill, before it is read. **(iii) Specificity.** Transplanting the eight highest-effect downstream positions recovers 0.74–0.79 of the decision; eight *random* downstream positions recover $\leq$0.035—a few specific aggregator tokens, not a diffuse code. **(iv) Writability.** Injecting a *false* note (the opposite conclusion's downstream KV) over an otherwise-consistent cache makes the decision follow the written note *against its own live field* (recovery ≈ 1.0); a handful of note tokens suffice. The account also holds off the synthetic template—field-only recovery stays $\approx$0 for multi-hop reasoning and free-form conversational phrasing, bounded only for near-verbatim attribute lookup, where the field is partly a copy (Appendix E).

Not a model-family artifact: five families. To rule out the aggregator-token account being a Qwen3/Llama tokenizer artifact, we replicate all five probes on two further architecture families, **Gemma-2-9B** and **Mistral-7B** (a tokenizer-robust readout; for Gemma-2 we keep its attention/logit soft-capping intact). Every result holds: field-only recovery 0.005/0.137 vs. full-downstream 1.0; the conclusion/content dissociation (trigger-only ≈ 0 with the field held identical vs. notes ≈ 1.0); top-8 vs. random-8 specificity (0.95 vs. 0.48); false-note injection (0.98–1.0, follow-rate 1.0); and write-before-read timing (write depth 0.19–0.26 vs. decision commit 0.47–0.48). The mechanism is consistent across *five* families (Qwen3, Llama-3.1, Gemma-2, Mistral; Appendix E).

4 Consequence I: the cache is editable

Figure 4 contrasts the two cache operations this section and the next make precise: editing appends a salient correction; composing repositions and splices precompiled KV.

Naive editing fails—by design. The mechanism predicts the in-place edit will fail, and it does: refreshing the field's KV while reusing the stale downstream leaves the decision at the old value (Figure 1b). The prefix before the field is genuinely reusable (deviation 0.0); the problem is that the conclusion was already memoized after it.

The robust fix is an erratum, not recomputation. If the stale notes carry an old conclusion, the cheapest correct intervention is to append a *salient erratum* that supplies the corrected state

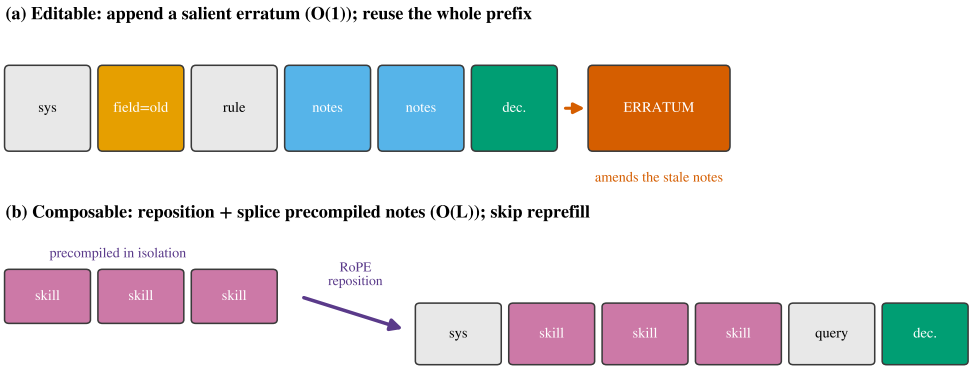

Figure 4: **Two cache operations.** (a) *Editable*: append a salient erratum that amends the stale notes ($O(1)$ extra work, the whole prefix is reused). (b) *Composable*: precompute a skill's KV in isolation, RoPE-reposition it to the target positions, and splice it in ($O(L)$, no reprefill).

late in the context—"`[STATE UPDATE] field` $\rightarrow$ `new; overrides any earlier value and conclusion`"—so the decision token attends to a fresh, authoritative note. Appended after the original field (`field+erratum`), this matches the strong *hoist-to-end* baseline's oracle correctness *without rewriting the prompt*: under chain-of-thought the erratum recovers the flipped decision at 0.97 on Qwen3-8B (Figure 5a), and the edit is append-only, so it composes with prefix caching (Section 8).

When the $\sim$1%-compute edit alone suffices: a reasoning/scale law. Surprisingly, under explicit reasoning the cheap in-place edit *by itself* can recover the decision, because the chain re-reads the (now-refreshed) field and re-derives the conclusion rather than trusting the stale notes. This recovery is strongly model-size dependent (Figure 5b): field-only reasoning recovery falls as the memoized conclusion becomes "stickier" with scale, so the minimal amount of selective recomputation needed, `field+selective@`K (refresh the field plus the K highest-effect downstream tokens), varies from $K^\star \approx 4$ at 8B to > 64 at 4B (Figure 5c). Without reasoning, no small K helps (0.00 recovery at every scale). `field+selective@`K is therefore a genuine but *unreliable* surgical tool—effective when the conclusion is not sticky, model-dependent otherwise; we report it honestly rather than as a default.

A frontier, not a winner. A controlled comparison (full table in the appendix) shows no single dominant method. Hoist-to-end is cheapest but demands prompt surgery and pre-identification of every field; `field+erratum` matches it with no surgery at a one-line append; `in_place` is near-free but only under reasoning; a KV-deviation-ranked selective recompute (CacheBlend-style) underperforms here because it chases changed keys rather than the tokens that *memoized the conclusion*. The practical recommendation is `field+erratum` as the robust default, with `in_place` as a free fast-path for reasoning models.

Why not edit the weights? A natural objection: to act on a changed field, why not edit the model's *weights* (ROME/MEMIT) or fine-tune, rather than the cache? We compare on the paper's gated task (status `pending`$\rightarrow$`shipped`, so *cancel*$\rightarrow$*deny*) against a faithful rank-one ROME—validated on the canonical factual edit first ("the Eiffel Tower is in" *Paris*$\rightarrow$*Rome*, locality intact), so the baseline is not crippled—and a LoRA fine-tune (Table 1). All of `field+erratum`, ROME,

Table 1: **KV editing vs. weight editing** for mutable per-request state (Llama-3.1-8B). Weight edits succeed at the target yet are global (contaminate concurrent requests), damage unrelated decisions, and are 30–50× slower per edit.

method	flips decision?	edit latency	cross-req. contamination	collateral (unrelated)
`field+erratum` (KV)	✓	114 ms	**0**	**0**
`in_place` (KV)	✗ (no CoT)	71 ms	0	0
ROME (rank-one)	✓	5.6 s + 11.6 s cov	1.0	0.5
LoRA fine-tune	✓	3.1 s	1.0	0.5

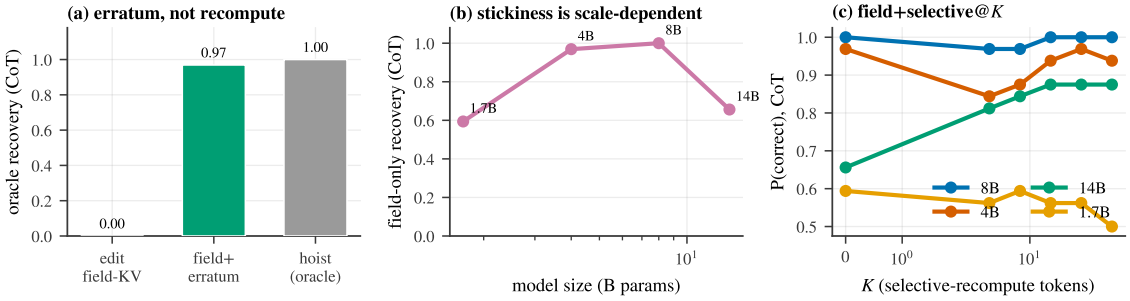

Figure 5: **Editing the cache.** (a) Editing the field's KV is ignored; `field+erratum` reaches the hoist-to-end oracle under CoT. (b) The memoized conclusion grows stickier with scale, so field-only reasoning recovery varies non-monotonically across model sizes. (c) `field+selective@`K: the minimal recompute $K^\star$ to reach full quality is model-dependent.

and LoRA *succeed* at flipping the target decision. But a weight edit is *global*: the same model instance can no longer hold `status=shipped` for one request and `pending` for another, so *all* concurrent orders that are genuinely still pending are wrongly flipped (cross-request contamination 1.0), and half of an unrelated decision battery drifts (0.5)—the ROME/fine-tune specificity tax—at 3–6 s per edit (plus a one-time covariance pass for ROME). The append-only erratum lives in a *per-sequence* cache: zero cross-request contamination, zero collateral, 114 ms, and it composes with prefix caching (Section 8). Weight editing targets *durable, global facts*; mutable per-request, per-turn state is the wrong job for it—which is precisely the niche the editable KV cache fills.

5 Consequence II: the cache is composable

From mechanism to prediction. If a region's notes are localized and re-derivable from context the decision can still see, then the notes should be *position-portable*: we can compute a reusable chunk's KV once, in isolation, move it to a new absolute position, and splice it in. Concretely we precompile a long *skill* (a policy or tool specification), and because the attention library caches post-RoPE keys, we re-rotate the chunk's keys from their source positions to the target positions (values are position-free) before concatenating (Figure 4b). This is the position-aware reuse of Liu et al. [11]; our point is that the *mechanism predicts it should preserve behavior*.

Transplant is behaviorally indistinguishable from full recompute. It does. The spliced skill matches a full reprefill in next-token logits with cosine 0.90–0.999 across the full model family— Qwen3-1.7B through 32B (including FP8 and the 30B-A3B Mixture-of-Experts), Gemma-2/3, Mistral-7B, Llama-3.1-8B and 70B, and DeepSeek-R1 (Figure 6b)—and on the models compe-

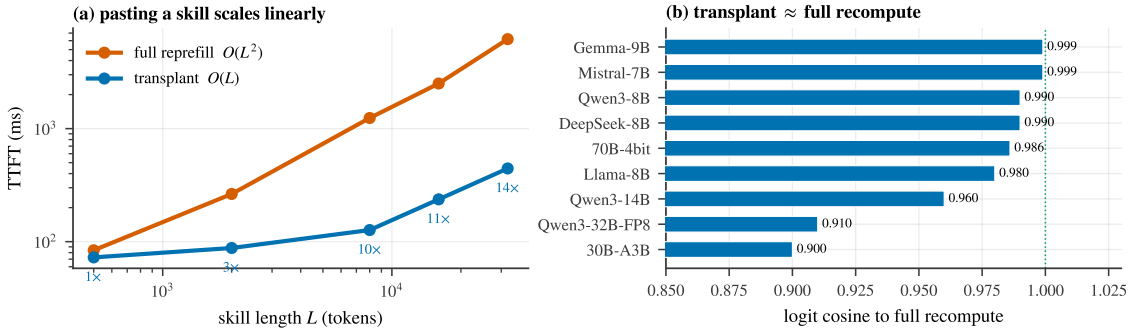

Figure 6: **Composing the cache.** (a) Pasting a precompiled skill is $O(L)$ vs. full reprefill's $O(L^2)$; TTFT speedup reaches 13.9× at 32k tokens. (b) The transplanted skill matches full recompute in next-token logits (cosine 0.90–0.999) across the model family.

tent at the task it preserves *correct* skill-following across 8 diverse domains and 3 families (24/24, cosine 0.98–0.999), including 16/16 under chain-of-thought.

Context-robustness and the seam. A chunk precompiled in isolation matches one that attended to the real preceding context, because the decision re-derives from context it still sees. The one residual error is a *seam* at the chunk's start—the first tokens that, in a full prefill, would have attended to the now-missing prefix. Recomputing a few boundary tokens (*seam-repair*) closes it; this is exactly the boundary recompute of CacheBlend/EPIC/MPIC, and Section 3 explains why it is the boundary, specifically, that needs repair.

Linear-time TTFT. Full reprefill of a length-L skill is $O(L^2)$; transplant is $O(L)$ (a re-rotation pass plus the suffix). Time-to-first-token speedups grow with skill length: 3× at 2k tokens, 9.8× at 8k, and 13.9× at 32k on an 8B model (Figure 6a). A library of skills composes (decisions preserved for $N = 1$–4 concurrent skills).

Generality of content, placement, and agency. Transplantation is not specific to rule-like skills. It preserves decisions for *facts/RAG* passages as well as rules; for chunks inserted in the system area *and* mid-trajectory as tool results; and—measured with actual function calls rather than a proxy—it preserves *agentic tool-calling* (N=108 with bootstrap CIs: function-call accuracy $1.00\,[1.0, 1.0]$ on Mistral, Llama-3.1, Qwen3-8B/32B-FP8/30B-A3B). The one consistent exception is sliding-window attention (Gemma), which we diagnose and fix in Section 7. Full per-domain scorecards are in the appendix.

6 One substrate: the keystone and a unified agent

The keystone: editing inside a transplant. If editing and composing are truly two operations on the same object, then editing a field that lives *inside a transplanted skill* should behave exactly as editing a field in a normally-prefilled context. We test this directly: transplant a skill whose body contains a mutable field, then apply each editing method to that field and measure recovery, comparing the *composed* cache (skill spliced in) against a fully *recomputed* cache. The editing mechanism reproduces verbatim (Figure 7a): the in-place edit is weak (≈ 0.05), selective recompute recovers (sel@32 ≈ 0.80), the erratum is strongest, and crucially *composed* $\approx$ *recomputed* for every method (points lie on the diagonal) across Gemma-2-9B and Llama-3.1-8B. The notes a transplant pastes in are the same notes an edit amends—one substrate.

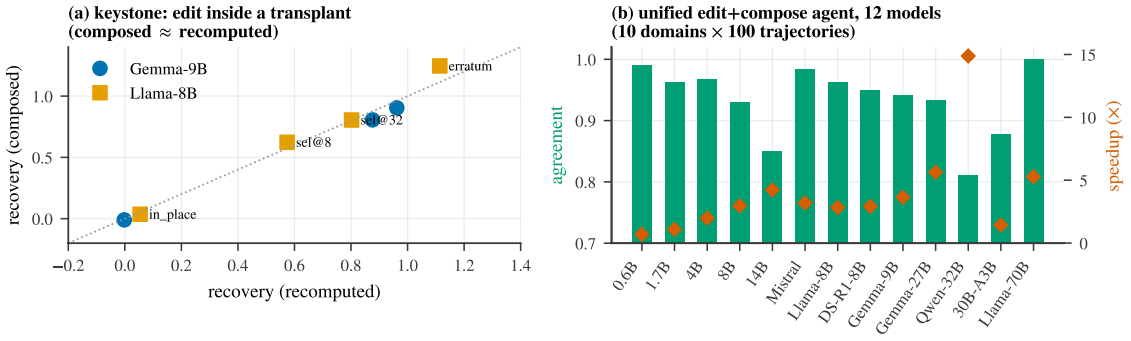

Figure 7: **Edit and compose are one substrate.** (a) Keystone: editing a field *inside* a transplanted skill reproduces the editing mechanism, and the *composed* cache matches the *recomputed* cache for every method (points on the diagonal). (b) A unified edit+compose agent over twelve models (10 domains × 10 instances, 300 decisions): unified-vs-full agreement (bars) and cumulative-TTFT speedup (markers).

A unified edit+compose agent. We embody both operations in a single live agent loop: a long policy is *composed* once and never re-prefilled; as the world changes across turns the mutable state is *edited* by appended errata; and each turn reuses the longest cached prefix and prefills only the delta. Against a reprefill-every-turn baseline, over 10 agent domains × 10 instances (300 decisions) per model and across twelve models, the unified path is decision-identical to full recompute with agreement 0.81–1.00 (e.g. Llama-3.1-8B 0.963, Mistral-7B 0.983, Llama-3.1-70B 1.00) at cumulative-TTFT speedups up to 14.9× (Figure 7b); the speedup scales with policy length × turns. This is editing and composing operating together, losslessly and faster, across the model family.

7 Reach and the 2026 frontier

Scale, quantization, MoE. The transplant is faithful from 0.6B to 32B, on FP8 checkpoints, on a 30B-A3B Mixture-of-Experts, and on a 4-bit 70B model (feasibility 8/8, logit cosine 0.986); the unified agent of Section 6 likewise spans all twelve.

Multimodal: images take notes too. In an agent trajectory an image costs a full prefill—the vision tower *plus* prefilling its >1k soft-tokens through the LM. Because image notes are also position-portable, we cache an image's LM-side KV once and splice it, re-running only text. Across 120 diverse VQA tasks per model (perception/reasoning/agentic), the spliced image is near-lossless versus full re-encode—agreement 0.958–1.0 on Qwen2.5-VL-3B/7B/32B and Qwen3-VL-8B (Figure 8b)—and reusing a cached image is 2.4–8.4× faster TTFT. Moving an image to a different trajectory position requires re-rotating only the temporal axis of M-RoPE; we handle both the *sectioned* (Qwen2.5-VL) and *interleaved* (Qwen3-VL) layouts, with position-shifted transplant lossless (agreement 0.99, Δ=161 positions). This subsumes MPIC's multimodal reuse [30], replacing its boundary-recompute with exact re-rotation.

The substrate is any per-token attention KV. The operations depend on the cache *representation*, not the attention kernel, so we map exactly where they hold (Figure 8a):

- **Free**—throughput optimizations that keep per-token KV: FlashAttention, paged attention / vLLM, and GQA/MQA (already used by every model we ran).
- **Adapter (implemented, validated)**—representation changes. *MLA* (DeepSeek-V2/Coder-V2)

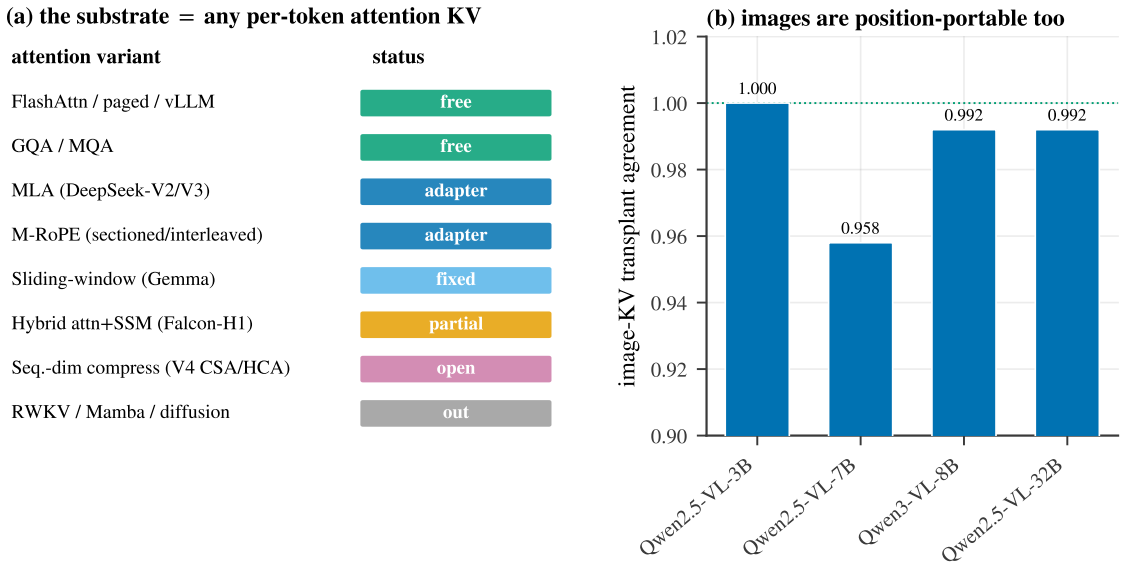

Figure 8: **Reach.** (a) The substrate is any per-token attention KV cache; we map each attention variant as free / adapter / fixed / partial / open / out-of-scope. (b) Image-KV transplant is near-lossless across vision-language models—images are position-portable too.

caches a position-free latent plus a small decoupled-RoPE sub-vector; our decoupled-`k_pe` reposition re-rotates only that sub-vector, giving logit cosine 0.98 and composed-vs-full agreement 1.00 on DeepSeek-Coder-V2-Lite. *Interleaved M-RoPE* (above) is the other adapter.

- **Fixed**—sliding-window (Gemma): the default cache truncates sliding layers to the window, breaking uniform splices beyond it; keeping the *full* per-token KV and letting the attention *mask* enforce the window restores correctness (the previously-failing unified agent now runs at agreement 0.93–0.94).
- **Partial**—hybrids with full-attention layers (Falcon-H1): the attention KV is transplantable but the per-layer Mamba scan-state is recurrent, not per-token, so a correct transplant must re-scan the Mamba path and saves only the attention fraction.
- **Open frontier**—sequence-dimension KV compression (DeepSeek-V4's CSA/HCA) merges tokens into sub-token-count entries, so edit/splice become block-granular; DeepSeek-V3.2's sparse attention (DSA) is MLA plus top-k selection and inherits our MLA adapter.
- **Out of scope**—no per-token attention KV: pure-recurrent (RWKV), pure-SSM (Mamba), and diffusion LMs. The prompt-level erratum still applies there, but it is not a KV operation.

8 Systems payoff

A real agentic environment. On the τ^2-bench retail environment—single tool-decisions and a multi-turn autonomous-agent loop scored by the environment's own tool enforcement—an agent that reuses a stale cache after a state change collapses, while `field+erratum` preserves task success at a fraction of the recompute. On the real $\sim$1.4k-token retail policy, transplant reproduces the clean decision (composed = full on Llama-3.1 and Mistral); the one hard case, a long buried field that must *flip* a conclusion, requires the robust `field+erratum` edit rather than transplant-plus-bare-

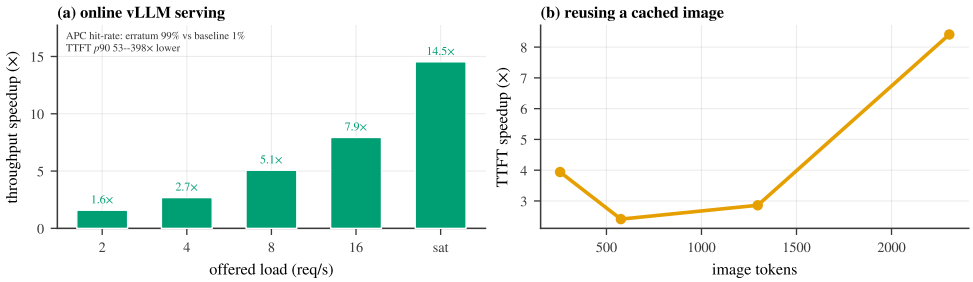

Figure 9: **Systems payoff.** (a) Online vLLM serving (V1 engine, CUDA graphs, continuous batching, APC, Poisson arrivals): the append-only erratum keeps the prefix cache-aligned (98.5% vs. 1% APC hit-rate), so its throughput advantage grows with offered load—up to 14.5× at saturation—while $p90$ TTFT is 53–398× lower. (b) Reusing a cached image (skipping the vision tower and image-token prefill) accelerates time-to-first-token, more so for larger images.

erratum—the same long-context lesson the editing axis predicts, now in a real environment.

A comprehensive online serving benchmark. Because the erratum is append-only, it composes with standard automatic prefix caching (APC): the static prefix stays cache-aligned and only the short erratum (and the decode) is new work, whereas writing the new value *into* the prefix changes a cached block's content hash and invalidates every downstream block. We test this on vLLM's V1 engine as a real online server—`AsyncLLMEngine` with CUDA graphs, continuous batching, APC, and *Poisson* request arrivals at controlled offered load (not an offline batch)—over a shared ∼8k-token agent policy with one mutable field, measuring TTFT percentiles, throughput, and the engine's own APC hit-rate (Figure 9a). The append-only edit keeps the prefix a cache hit (98.5% **vs.** 1.0% **APC hit-rate**); the in-prefix baseline is prefill-bound and *saturates at* ≈1.5 *req/s*, so its $p90$ time-to-first-token collapses under load (22–55 s) while the erratum stays 86 ms–1 s (53–398× lower TTFT). The throughput advantage *grows with offered load*—1.6× at 2 req/s up to **14.5**× at saturation—exactly as predicted for a compute-bound vs. cache-bound regime. The image-KV reuse of Section 7 contributes a complementary serving win—2.4–8.4× faster first-token as image size grows (Figure 9b)—by skipping the vision tower and image-token prefill entirely.

9 Application: editable and composable user memory

A concrete, high-value instance of the edit+compose substrate is *user memory*: the large, dynamically-summarized profile of facts an assistant re-reads every turn. Memory is big (thousands–tens of thousands of tokens), reused across turns, and *mutated mid-session* by tool calls—so where it lives in the prompt is a dilemma the mechanism of Section 3 explains. Placed at the *front* (*[sys][MEM][traj]*), the trajectory memoizes memory-conditioned conclusions downstream, so a memory change forces a costly downstream reprefill (and an in-place edit is ignored). Placed at the *end* (*[sys][traj][MEM]*), memory's KV depends on the whole trajectory and must be re-attended every turn. We resolve this by treating memory as a *skill that is also edited*: precompile it once in isolation, place it late (*[sys][traj][MEM][query]* so the decode reads it directly), RoPE-reposition it each turn, repair one boundary token, and edit it in place when it changes.

Memory transplant is faithful, to 70B (E2). Across ten models from 0.6B to 70B, a precompiled+repositioned memory chunk reproduces the full-recompute decision logits with cosine 0.94–0.9996; a *single* seam-repair token closes the start-of-chunk boundary gap (Llama-3.1-8B late: cos 0.94 → 0.994). The cleanest decision-governance test is Llama-3.1-70B, whose decisions

genuinely vary (so agreement is non-trivial, unlike the constant-answer regime of smaller models): the late, seam-repaired transplant reproduces the full-recompute decision 0.93 of the time at logit cosine 0.997, and *late placement beats early* (0.93 vs. 0.83) exactly as the mechanism predicts (the decode reads memory directly rather than through pre-digested notes across the transplant boundary). The no-rotation control collapses (late decision agreement 0.18 vs. 0.78 rotated), confirming the re-rotation is necessary.

Memory is editable mid-session (E3). When a stored fact is toggled, reusing the *stale* memory recovers the flipped decision essentially never (≤ 0.03). Every real edit recovers it, and—consistent with Section 4—under chain-of-thought the *near-free in-place edit* (one token recomputed) suffices, and *strengthens with scale*: correctness $0.92 \to 0.99 \to 1.00$ across Qwen3-1.7B/4B/14B (and 0.98 at 32B). On models where the chain does not re-read the field, the append-only ERRATUM is the robust fallback (McNemar ERRATUM>`in_place` on Llama-3.1-8B, $p = 0.031$). This is the editing axis that concurrent KV-cache memory systems lack: MemArt [1] and EPIC [5] retrieve and splice *static* memory blocks position-independently but have no in-place memory *update*; our additions are the editing operation, the decision-governance lens, and the mechanism (Section 3) that explains why boundary recompute suffices.

Keystone: editing inside transplanted memory reproduces the mechanism. Editing a field *inside a transplanted memory chunk* in direct mode (Llama-3.1-70B, whose decisions vary so recovery is measurable) reproduces Section 3's memoization verbatim: refreshing the field's KV alone recovers only 0.55 of the flipped decision (the conclusion was memoized *downstream*, not in the field), and recovery climbs monotonically as more downstream tokens are recomputed ($0.55 \to 0.84$ at $K{=}16 \to 0.94$ full), exactly as for a normally-prefilled context. Under chain-of-thought this stickiness *dissolves*—a selective@K sweep is flat at $\approx$0.98 for all K (the chain re-reads the field, so $K^\star \approx 1$). Edit and compose thus act on one substrate, and the direct/CoT dissociation matches the editing law of Section 4.

Real memory: LoCoMo external validity. The synthetic gated decisions isolate the mechanism; to test real memory we run the long-conversation QA benchmark LoCoMo [14] (MemArt's setting): the multi-session dialogue (median $\sim$19.7k tokens) is the memory, precompiled and spliced before each question. Over *all* 1,540 answerable questions per model, transplant is *statistically equivalent* to full recompute in QA accuracy on Qwen3-4B, 14B, and 32B (TOST, margin 0.03: $|\Delta| \leq 0.015$) and within a small -2.7 points on Llama-3.1-8B, with answer-token logit cosine 0.991–0.998 throughout. The *compose* axis thus holds on real conversational memory, not only synthetic decisions.

Granularity is a free knob (E4). Splitting memory into S independently-precompiled blocks makes a localized edit $S\times$ cheaper (recompute one block) and is *decision-lossless* up to $S{=}16$ (agreement flat at 1.00 on Qwen3-4B), because the independent facts are integrated at read time, not within memory. Logit cosine degrades with S ($0.998 \to 0.953$) but *identically whether the gating facts are contiguous in one block or split across blocks*—splitting relevant facts across independently-precompiled blocks does not specifically hurt; only genuinely cross-*referential* facts (untested) should.

Placement and the pre-digestion cost (E1). Pre-digestion is real, and we quantify it. Under full recompute and chain-of-thought, reading memory *late* carries a small but *statistically significant* accuracy cost vs. *early* (Qwen3-4B, $N{=}192$: GEE-logistic $\beta_{\text{late}} = -0.44$, $p = 0.018$), and the cost *grows with memory length*: the early$-$late gap is $\approx$0 at 2k tokens but $+0.09$ at 16k and $+0.16$ at 32k. The mechanism explains it—late placement forgoes prefill-time pre-digestion and relies on the

decode/CoT to integrate raw memory. This is a *tradeoff, not free*: late placement is still preferred because it enables $O(L)$ editing/transplant and 2.3–4.3× TTFT (below) and the transplant is faithful at a fixed placement (above), so the net is small accuracy for large efficiency—with early placement preferable when memory is very long and accuracy-critical. (Direct one-shot decisions are at chance for the reasoning-native Qwen3 family *to 32B*, breaking only at Llama-3.1-70B, direct 0.81; CoT is the operative regime across scale—see appendix.)

End-to-end agent (E5). We implement a live agent (Appendix D) that composes memory once, re-rotates it each turn, and edits it on tool-driven changes. Over 24 sessions × 12 turns with ≈2k-token memories, against a reprefill-every-turn-at-the-end baseline it cuts cumulative time-to-first-token by 2.3–4.32× (growing with model size, to 32B), and against front-placement reprefill-on-change by 1.4–3.27×, while reproducing the full-reprefill next-token logits at a token-matched oracle (cosine 0.98–0.99). One honest caveat: greedy chains-of-thought are sensitive to sub-percent logit differences, so the exact reasoning *chain* is not always reproduced (chain agreement 0.31–0.69) even though the next-token decision is faithful and CoT *accuracy* is comparable; the clean decision-governance equivalence is the short-context editing result above (Section 4-style, agreement 0.89–0.95). User memory is thus a second instance of one substrate that is both editable and composable.

10 Limitations

Our operations require a per-token attention KV cache, so pure-recurrent, pure-SSM, and diffusion models are out of scope, and hybrid attention+SSM models are only partially served (the recurrent state is not transplantable; Section 7). The MLA and sliding-window adapters are validated but carry residual edge cases: a single transplanted *chunk that itself exceeds the sliding window* drops to logit cosine ≈ 0.89 (real skills are far smaller), and our small MLA checkpoints ship legacy-cache custom modeling that we shim. Sequence-dimension KV compression (DeepSeek-V4-class) and cross-attention image caches are open: the unit of edit/transplant there becomes a compressed block rather than a token, which we have analyzed but not implemented. The `field+selective@`K surgical edit is unreliable (model- and domain-dependent stickiness) and we present it as such, not as a default. Finally, while several studies use synthetic policies for controlled measurement, we mitigate this with the real τ^2-bench retail policy and real images; broader real-workload evaluation remains future work. The caching machinery we build on is prior art (Section 2); we claim the mechanism, the unification, the decision-governance lens, and the attention-variant adapters.

11 Conclusion

A surgical edit to a field's KV is ignored not because the cache is fragile but because the model already did the work: at prefill it memoizes the field-conditioned *conclusion* onto downstream aggregator tokens, and the decision reads those notes. Reframing the KV cache as a notebook of memoized conclusions makes two capabilities fall out of one mechanism—you can *edit* the notes (amend them with a late, salient erratum rather than recompute) and *compose* the notes (reposition and splice a precompiled skill, in $O(L)$ time)—and a keystone experiment shows they act on a single substrate. The substrate is any per-token attention KV cache: it holds across scale, quantization, Mixture-of-Experts, and multimodal image caches, transfers freely across throughput optimizations, and reaches Multi-head Latent Attention, sliding-window, and interleaved-M-RoPE models through small adapters—up to the edge of the 2026 sparse/compressed-attention frontier, where the unit of editing and composition becomes a block rather than a token. We hope the "models take notes at prefill" lens is useful beyond editing and composition: the cache is a readable, writable record of what a model has already concluded.

References

[1] Anonymous. MemArt: KVCache-centric memory for LLM agents, 2026. Under review (OpenReview).

[2] DeepSeek-AI. DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434*, 2024.

[3] DeepSeek-AI. DeepSeek-V3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.

[4] In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. Prompt cache: Modular attention reuse for low-latency inference. In *Proceedings of Machine Learning and Systems (MLSys)*, 2024.

[5] Junhao Hu et al. EPIC: Efficient position-independent caching for serving large language models. In *International Conference on Machine Learning (ICML)*, 2025.

[6] Gabriel Ilharco, Marco Tulio Ribeiro, Mitchell Wortsman, Ludwig Schmidt, et al. Editing models with task arithmetic. In *International Conference on Learning Representations (ICLR)*, 2023.

[7] Chao Jin, Zili Zhang, Xuanlin Jiang, Fangyue Liu, et al. RAGCache: Efficient knowledge caching for retrieval-augmented generation. *arXiv preprint arXiv:2404.12457*, 2024.

[8] Kenneth Li, Oam Patel, Fernanda Viégas, Hanspeter Pfister, and Martin Wattenberg. Inference-time intervention: Eliciting truthful answers from a language model. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[9] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, et al. SnapKV: LLM knows what you are looking for before generation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[10] Jack Lindsey et al. On the biology of a large language model. *Transformer Circuits Thread, Anthropic*, 2025.

[11] Yang Liu, Junhao Hu, Yunfei Gu, et al. CacheSlide: Unlocking cross position-aware KV cache reuse for accelerating LLM serving. In *USENIX Conference on File and Storage Technologies (FAST)*, 2026.

[12] Yuhan Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, et al. CacheGen: KV cache compression and streaming for fast large language model serving. In *Proceedings of ACM SIGCOMM*, 2024.

[13] Zichang Liu, Aditya Desai, Fangshuo Liao, Weitao Wang, et al. Scissorhands: Exploiting the persistence of importance hypothesis for LLM KV cache compression at test time. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[14] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of LLM agents. In *Proceedings of the Association for Computational Linguistics (ACL)*, 2024.

[15] Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in GPT. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[16] Kevin Meng, Arnab Sen Sharma, Alex Andonian, Yonatan Belinkov, and David Bau. Mass-editing memory in a transformer. In *International Conference on Learning Representations (ICLR)*, 2023.

[17] Bowen Peng, Jeffrey Quesnelle, Honglu Fan, and Enrico Shippole. YaRN: Efficient context window extension of large language models. In *International Conference on Learning Representations (ICLR)*, 2024.

[18] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, et al. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[19] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568, 2024.

[20] Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han. Quest: Query-aware sparsity for efficient long-context LLM inference. In *International Conference on Machine Learning (ICML)*, 2024.

[21] Eric Todd, Millicent Li, Arnab Sen Sharma, Aaron Mueller, Byron C Wallace, and David Bau. Function vectors in large language models. In *International Conference on Learning Representations (ICLR)*, 2024.

[22] Jesse Vig, Sebastian Gehrmann, Yonatan Belinkov, et al. Investigating gender bias in language models using causal mediation analysis. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[23] Kevin Wang, Alexandre Variengien, Arthur Conmy, Buck Shlegeris, and Jacob Steinhardt. Interpretability in the wild: a circuit for indirect object identification in GPT-2 small. In *International Conference on Learning Representations (ICLR)*, 2023.

[24] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations (ICLR)*, 2024.

[25] Jingbo Yang et al. KVLink: Accelerating large language models via efficient KV cache reuse. *arXiv preprint arXiv:2502.16002*, 2025.

[26] Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. CacheBlend: Fast large language model serving for RAG with cached knowledge fusion. In *Proceedings of the European Conference on Computer Systems (EuroSys)*, 2025.

[27] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, et al. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

[28] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*, 2024.

[29] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, et al. H2O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[30] Shiju Zhao, Junhao Hu, Rongxiao Huang, Jiaqi Zheng, and Guihai Chen. MPIC: Position-independent multimodal context caching system for efficient MLLM serving. *arXiv preprint arXiv:2502.01960*, 2025.

A Models evaluated

Table 2 lists the models used across the paper. All runs are on a single RTX PRO 6000 (Blackwell, 96 GB); FP8 and 4-bit checkpoints are the official quantized releases.

Table 2: Model zoo. "role" indicates the experiments a model appears in.

model	family / attention	precision	role
Qwen3-0.6B/1.7B/4B/8B/14B	Qwen3 (GQA)	bf16	mechanism, edit, compose, agent
Qwen3-32B	Qwen3 (GQA)	FP8	compose, agent, scale
Qwen3-30B-A3B	Qwen3 MoE (GQA)	bf16	compose, agent, MoE
Llama-3.1-8B / 70B	Llama (GQA)	bf16 / 4-bit	mechanism, edit, compose, agent
Mistral-7B	Mistral (GQA)	bf16	mechanism, compose, agent
Gemma-2-9B / Gemma-3-27B	Gemma (sliding-window)	bf16	mechanism, compose, agent, sliding-window fix
DeepSeek-R1-Distill-Llama-8B	Llama (GQA), reasoning	bf16	edit, compose, agent
DeepSeek-V2-Lite / Coder-V2-Lite	DeepSeek (MLA)	bf16	MLA adapter
Falcon-H1, Falcon-Mamba	hybrid / pure SSM	bf16	architecture boundary
Qwen2.5-VL-3B/7B/32B, Qwen3-VL-8B	VL (M-RoPE)	bf16	multimodal image-KV

B Editing: the baseline frontier and the K-sweep

Figure 10 plots the cost/correctness frontier behind Section 4: there is no single dominant method. `field+erratum` and the bare ERRATUM reach full correctness at ∼5–13% recompute with no prompt surgery; hoist-to-end matches them but rewrites the prompt; the in-place edit and a KV-deviation-ranked CacheBlend-style recompute are cheap but incorrect on these gated decisions. Numeric values are in Table 3. Figure 11 gives the full `field+selective@`K sweep across seven models, making the model-dependence of the minimal recompute explicit: small K suffices for the Qwen3 family but not for several others—the tool is effective but unreliable.

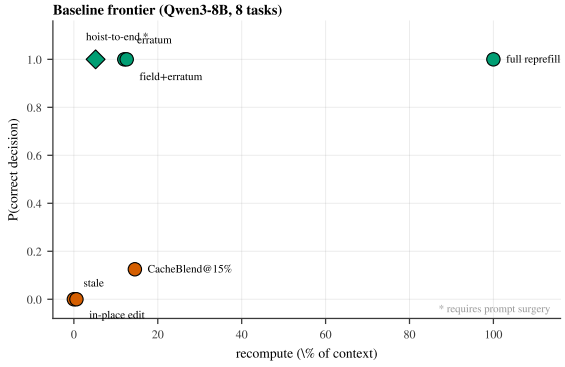

Figure 10: Cost/correctness frontier.

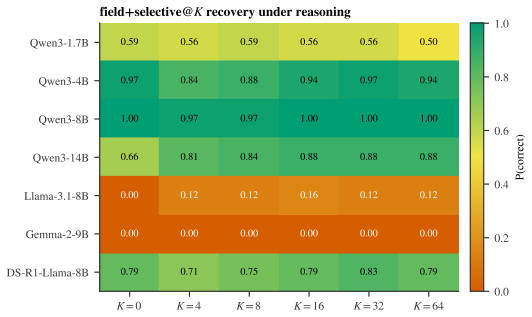

Figure 11: `field+selective@`K across models.

Figure 12 shows that the editing fix is an *attention-architecture* method: the erratum recovers the decision under reasoning on attention (GQA) and sliding-window backbones, partially on a hybrid attention+SSM model, and weakly on a pure SSM whose recurrent state has no per-token look-back.

C Composing: per-domain scorecards, multimodal, and the MLA adapter

Figure 13 is the composable scorecard: decision agreement between a transplanted skill and full recompute, by model and content type (facts in two insertion points, and agentic tool-calling). Standard attention models are at or near 1.0; the sliding-window Gemma models are the consistent

Table 3: Baseline frontier (Qwen3-8B, 8 gated tasks). "surgery" = requires rewriting the prompt.

method	P(correct)	recompute	prompt surgery
full reprefill	1.00	100%	no
hoist-to-end	1.00	5.2%	**yes**
`field+erratum`	1.00	12.6%	no
ERRATUM	1.00	12.0%	no
CacheBlend@15%	0.13	14.5%	no
`in_place` edit	0.00	0.6%	no
stale (reuse)	0.00	0%	no

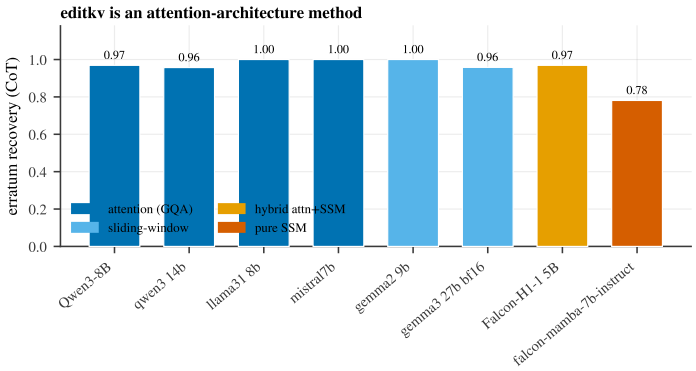

Figure 12: Erratum recovery under reasoning across attention, sliding-window, hybrid, and pure-SSM backbones.

exception (addressed in Section 7). Figure 15 breaks the multimodal result down by task category, and Figure 14 reports the MLA decoupled-`k_pe` adapter fidelity and the transplant TTFT speedup across models.

D The user-memory agent

The agent of Section 9 keeps the layout `[system][trajectory][MEMORY][query]`. The system prompt is prefilled once; the trajectory grows by cached deltas; the user-memory chunk is precompiled once in isolation and RoPE-repositioned to float just before the query each turn (an $O(L_{\text{mem}})$ re-rotation, no re-prefill) with one boundary token repaired; a memory change is applied either by recompiling the isolated chunk ($O(L_{\text{mem}})$, once) or by appending a salient erratum into the cached trajectory stream (append-only, composes with prefix caching). Confirmatory protocol: gated decisions whose governing facts live in a Markdown memory, balanced labels, chain-of-thought as the competent regime (direct one-shot memory-gated decisions are at chance for $\leq$8B models), with cluster-bootstrap CIs (10^4, persona-level), TOST equivalence (δ=0.03; cosine ≥ 0.98), GEE-logistic (cluster-robust), McNemar, and BH-FDR. Faithfulness is read against a token-matched full-reprefill oracle. Pre-registered hypotheses, margins, and the inclusion gate (oracle accuracy ≥ 0.80) are released with the code.

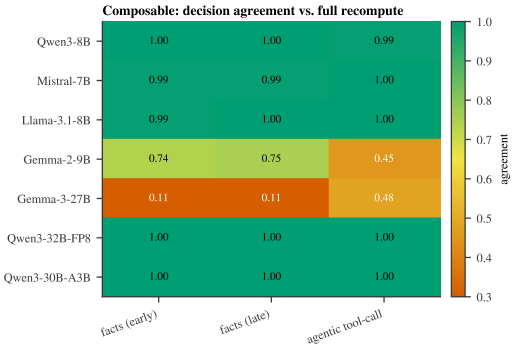

Figure 13: Composable agreement by model × content type.

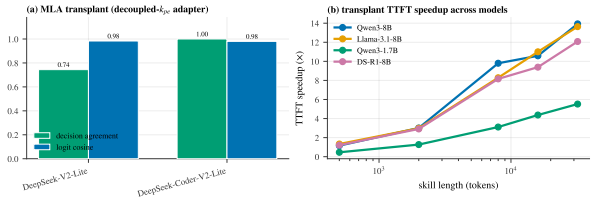

Figure 14: MLA adapter fidelity (a) and transplant TTFT speedup across models (b).

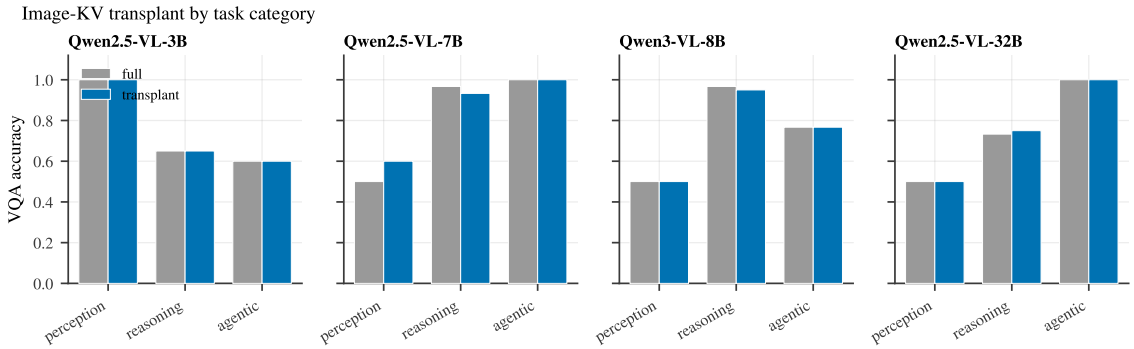

Figure 15: Image-KV transplant by task category (perception / reasoning / agentic), full re-encode vs. transplant, across four vision-language models. The transplant tracks full accuracy category-by-category.

E Deep mechanism controls: conclusion vs. content, timing, specificity, writability

These four controls (and an off-template generalization) tighten Section 3 from *decodability* to *causation, timing, and write-access*. Each runs on three models (Qwen3-8B, Qwen3-4B, Llama-3.1-8B); scripts `esys/mechd_*.py`, records `results/mechd_*`.

(i) Dissociation (Table 5, left). A polarity-parameterized rule names a single *trigger* value that selects the safe action, so flipping the trigger inverts the conclusion while the field value is byte-identical across the pair (the two prompts differ in exactly one token, inside the rule). Transplanting the post-trigger *notes* from the opposite-conclusion cache carries essentially the entire flipped conclusion, whereas patching the differing rule token carries none—with the field held constant, the decision is reading a memoized conclusion, not field content. A logit-probe finds both the conclusion and the field identity linearly decodable from the same downstream delimiter, so decodability cannot itself adjudicate; the causal transplant is required.

(ii) Timing. Using `output_hidden_states` on the prefill, the conclusion becomes linearly decodable (group-CV logistic probe, conclusion $\perp$ field by the 2×2 design) on the downstream aggregator at layer-depth 0.39/0.39/0.31 (Qwen3-8B/4B, Llama-3.1-8B), while the decision token's

logit-lens margin reaches its final sign only at depth 0.75/0.77/0.73—the note is written ∼12 layers before it is read, within one prefill.

(iii) Specificity (Table 5, right). Ranking downstream positions by individual transplant effect, the top-8 recover 0.74–0.79 of the decision; 8 random downstream positions recover ≤0.035. The conclusion sits on a few specific aggregator/delimiter tokens, not diffusely.

(iv) Writability. Overwriting an otherwise-consistent cache's downstream notes with the *opposite* conclusion's notes (the field token and prefix left intact and still implying the original answer) drives the decision to the injected conclusion: continuous recovery 0.99/0.98/1.02, with the top-8 note positions already flipping the belief in most instances. Editing (Section 4) is the benign use of this same write-access.

Off-template generalization. Re-running field-only vs. full-downstream recovery on (a) a 2-hop lookup, (b) free-form conversational phrasing, and (c) attribute lookup: field-only recovery is ≈0 for (a) and (b) across models (multi-hop −0.012 to +0.006; natural −0.012 to +0.054) with full-downstream 1.00, confirming the mechanism is not a template artifact; for near-verbatim attribute lookup (c) the field is partly a copy and carries 0.25–0.63, bounding the claim to *derived* conclusions.

Cross-family replication (Gemma-2, Mistral). To rule out a Qwen3/Llama tokenizer artifact we re-ran all five probes on **Gemma-2-9B** and **Mistral-7B** with a tokenizer-robust readout (space-prefixed single-token `cancel`/`deny`; Gemma-2's soft-capping kept intact, as the attention-knockout hook used only by the original circuit-knockout probe otherwise corrupts it), on the gated cancel/deny task (n=18 primary, 18 dissociation each; Table 4). All five replicate: field-only ≈ 0 vs. full-downstream 1.0; dissociation trigger-only ≈ 0 vs. notes ≈ 1.0; top-8 ≫ random-8; injection ≈ 1.0; and write-before-read timing. Two honest notes: Mistral's field-only recovery (0.137) is slightly above zero (still far below full-downstream 1.0), and random-8 specificity runs higher on this task (0.48–0.53 vs. ≈0.02 on the Qwen/Llama tool-call task) though top-8 (≈ 0.95) still dominates.

Table 4: Cross-family replication of the five deep-mechanism probes (Gemma-2-9B, Mistral-7B). Recovery toward the target conclusion; bootstrap means.

model	field-only	full-down	trigger-only	notes	top-8 / rand-8	write/commit depth
Gemma-2-9B	0.005	1.0	−0.00	1.0	0.96/0.48	0.26/0.48
Mistral-7B	0.137	1.0	0.001	0.995	0.94/0.53	0.19/0.47

Table 5: Deep mechanism controls (recovery toward the target conclusion; three models). Left: dissociation—patching the differing rule token vs. the downstream notes, field held identical. Right: specificity—top-k vs. random-k downstream positions, matched count.

(i) Dissociation			**(iii) Specificity**				
model	trigger only	notes	model	top-8	rand-8	top-16	rand-16
Qwen3-8B	−0.007	1.004	Qwen3-8B	0.78	0.009	0.92	0.06
Qwen3-4B	−0.007	1.009	Qwen3-4B	0.74	0.035	0.86	0.04
Llama-3.1-8B	+0.007	0.998	Llama-3.1-8B	0.79	0.005	0.86	0.04

F Online serving and weight-editing methodology

Online vLLM serving (Figure 9a). vLLM V1 `AsyncLLMEngine`, CUDA graphs enabled (*not* `enforce_eager`), continuous batching, automatic prefix caching on, `bfloat16`, `gpu_memory_utilization`= 0.85. Workload: a shared ∼8,066-token policy (real τ^2-bench retail policy plus neutral padding) with one mutable field; 96 requests per arm, 64 output tokens each. Requests arrive as a *Poisson* process at offered rates $\{2,4,8,16\}$ req/s and unthrottled (saturation). We timestamp first-token and completion per request in the client loop (TTFT, TPOT, end-to-end) and read the APC hit-rate per arm from the engine's Prometheus counters (`vllm:gpu_prefix_cache_{hits,queries}`). The in-prefix baseline writes the new field value early (invalidating downstream APC blocks); the erratum keeps the old prefix and appends the update. Headline: throughput speedup grows $1.6\times \to 14.5\times$ as load rises to saturation; $p90$ TTFT 53–398$\times$ lower; APC hit-rate 98.5% vs. 1.0%.

Weight editing (Table 1). ROME is implemented from scratch (uncentered key covariance $C=\mathbb{E}[kk^\top]$ at a mid MLP layer over a text sample, an optimized value $v^\star$, and the closed-form rank-one update of `down_proj`) and *validated on the canonical factual edit* ("the Eiffel Tower is in" *Paris*→*Rome*, locality preserved) before use, so the baseline is faithful. LoRA fine-tunes (r=8, `q,v,down` projections) on the stale-context decision until it flips. All methods are evaluated on the same gated decision; cross-request contamination is measured over 8 held-out orders that are genuinely still pending (correct = cancel), and collateral over a 10-item battery of unrelated single-token gated decisions.

G Reproducibility

All numbers are from local runs; result records and figure-generation scripts are released with the code. Bootstrap confidence intervals use 10^4 resamples. The unified-agent study uses 10 domains $\times$ 10 instances (300 decisions) per model; the multimodal and agentic studies use N=120 and N=108 respectively with bootstrap CIs. Mechanism cross-family replication (`esys/mechd_replicate.py`), online serving (`esys/vllm_serving_online.py`), and weight editing (`esys/rome.py`, `esys/weight_editing_comp` are released with their result JSONs.